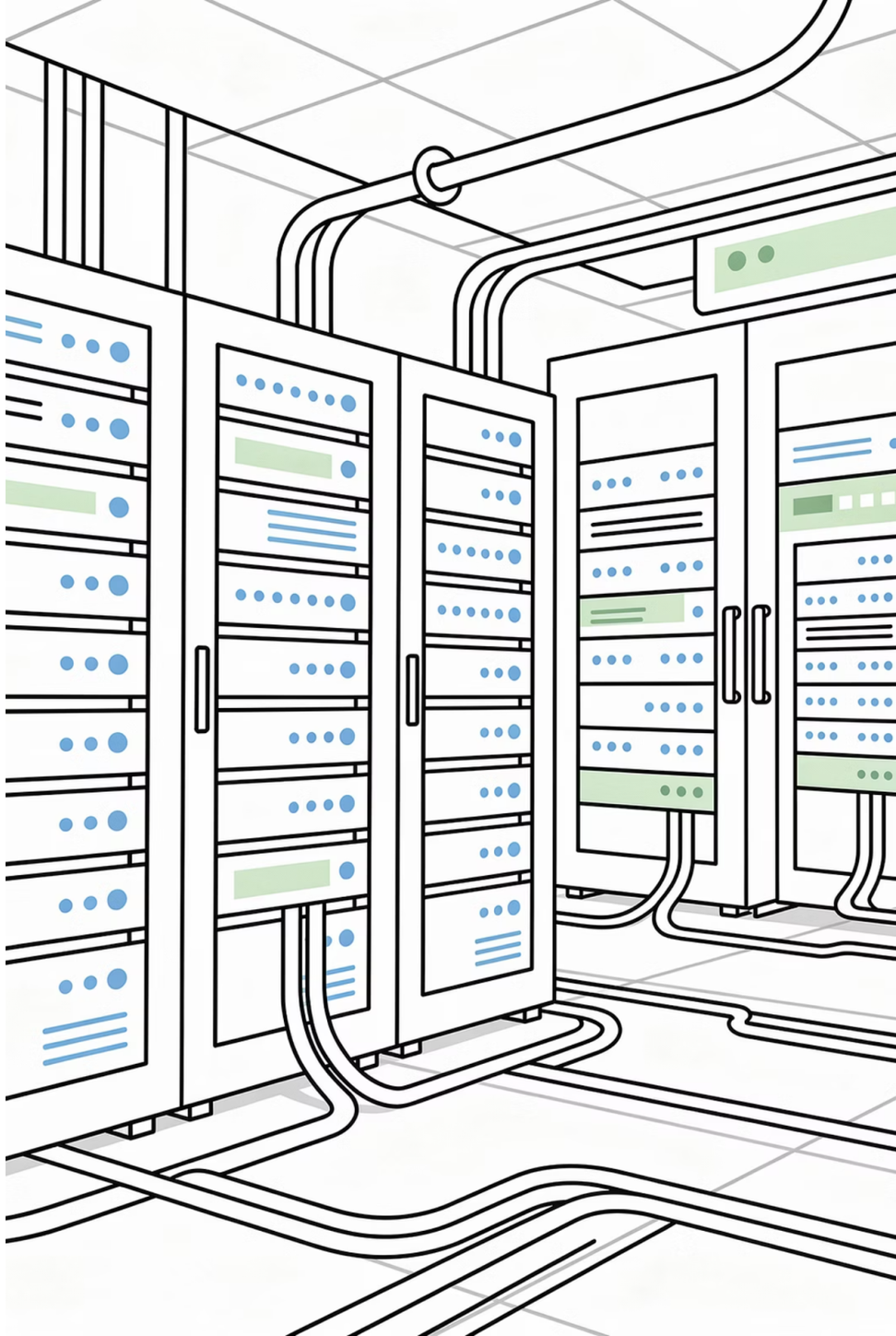
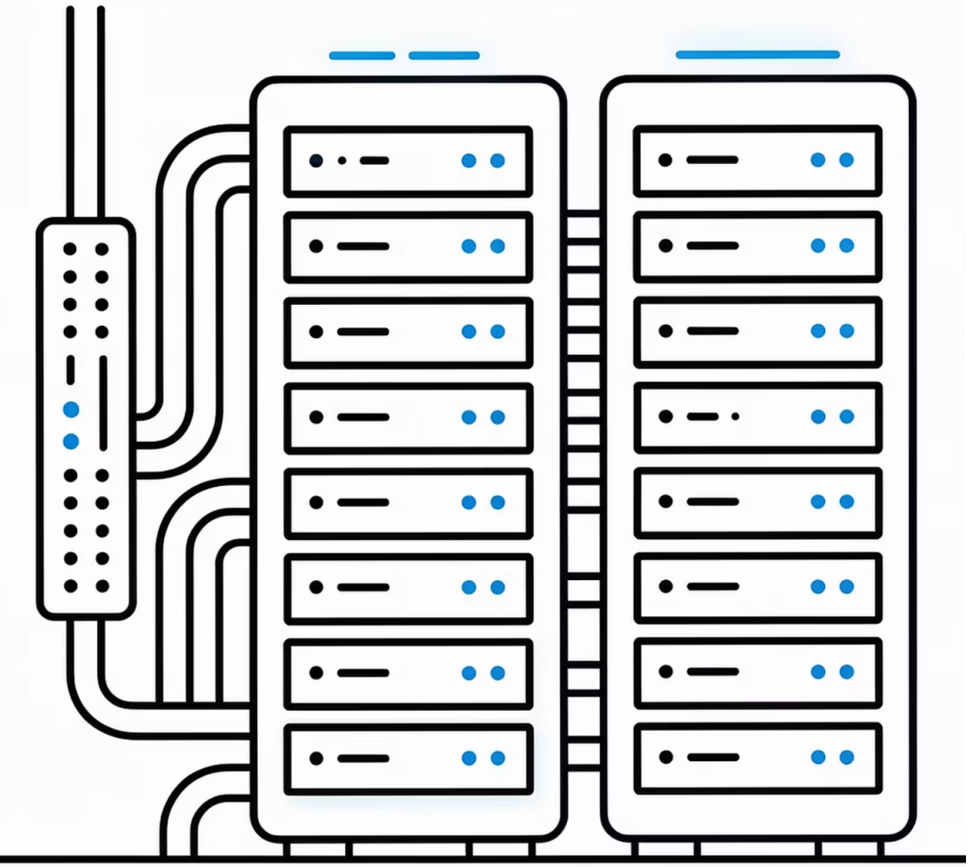
Өгөгдлийн сангийн удирдлага

F.ITM237

Мэдээллийн технологийн тэнхим

С.Өлзийбаяр (Ph.D)

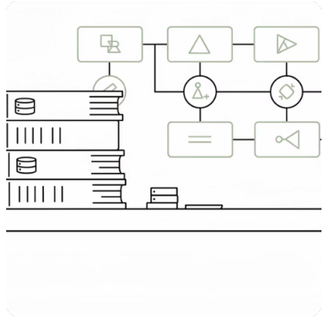




Өгөгдлийн сангийн дизайн ба өгөгдлийн загварчлал: Уламжлалтаас орчин үе рүү

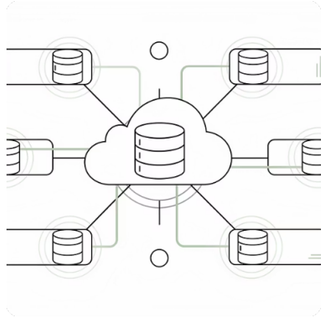
Мэдээллийн системийн оюутнуудад зориулсан өгөгдлийн сангийн архитектурын хувьслын цогц судалгаа. Сонгодог харилцааны дизайныг орчин үеийн NoSQL аргуудтай харьцуулж, хоёр аргын давуу ба сул талыг тодорхойлно.

Хичээлийн тойм



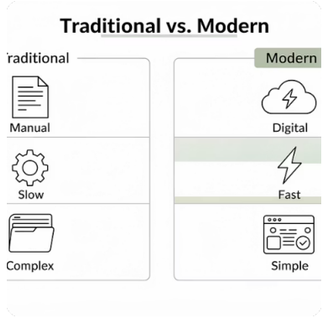
Уламжлалт үндэс

1970-аад оноос хойш өгөгдлийн сангийн дизайныг удирдаж ирсэн ER загварчлал болон нормчлалын зарчмуудыг эзэмших



Орчин үеийн хандлага

Өнөөгийн өргөтгөх чадвар болон уян хатан байдлын шаардлагыг хангах NoSQL болон Agile аргачлалуудыг судлах



Харьцуулсан шинжилгээ

Аль аргыг хэзээ хэрэглэх, бодит системд хэрхэн үр дүнтэй хослуулахыг ойлгох



Практик туршлага

Уламжлалт болон орчин үеийн аргачлалыг ашиглан өгөгдлийн сан зохион бүтээх практик туршлага хуримтлуулах

ТҮҮХ

Өгөгдлийн сангийн түүх ба хөгжил

1

1960-аад он

Анхны өгөгдлийн сангийн системүүд бий болсон үе. IBM-ийн IMS болон CODASYL загвар гарч ирэв

2

1970-аад он

Эдгар Кодд холбоос загварыг санал болгож, SQL хэл хөгжиж эхэлсэн

3

1980-1990-ээд он

Oracle, MySQL, PostgreSQL зэрэг арилжааны DBMS-ууд зах зээлд гарч ирэв

4

2000-аад он

NoSQL өгөгдлийн сангууд гарч ирж, том өгөгдөл боловсруулах шинэ эрин эхэлсэн

5

Орчин үе

Cloud-д суурилсан, AI интеграцтай, өндөр хүчин чадалтай системүүдийн хувьсал

Өгөгдлийн сангийн технологийн хувьсал



Өгөгдлийн сангийн дизайн нь өгөгдлийн хэмжээ, програмын шаардлага, архитектурын парадигмын өөрчлөлтөөс шалтгаалан үндсэн өөрчлөлтүүдийг туулж ирсэн. Энэхүү хувьслыг ойлгох нь яагаад олон өгөгдлийн сангийн парадигм зэрэгцэн оршиж, тус бүр нь хэзээ илүү давуу талтай болохыг ойлгоход тусална.

Түүхэн үе шатууд

- **Холбоост ӨС эрин үе:** Тогтмол схем, ACID гүйлгээ
- **Вэб хувьсгал:** Вэб-масштабын хурд шаардлагатай болсон
- **NoSQL эрин үе:** Том өгөгдөлд зориулсан уян хатан загвар
- **Polyglot Persistence:** Хэрэглээний тохиолдол бүрт өгөгдлийн сангийн холимог ашиглалт



Холбоост ӨС эрин үе: *1970–1980-аад он*

Edgar Codd-ын инноваци

Холбоост загвар (1970) нь өгөгдлийн менежментэд математик нарийвчлалыг нэвтрүүлж, өнөөг хүртэл оршин тогтнож байгаа үндэс суурийг тавьсан

SQL-ийн гарал

Стандартчилсан асуулгын хэл нь зарлагч өгөгдөлд хандах боломжийг бүрдүүлж, хөгжүүлэгчид өгөгдлийн сантай харилцах арга замыг хувьсгасан

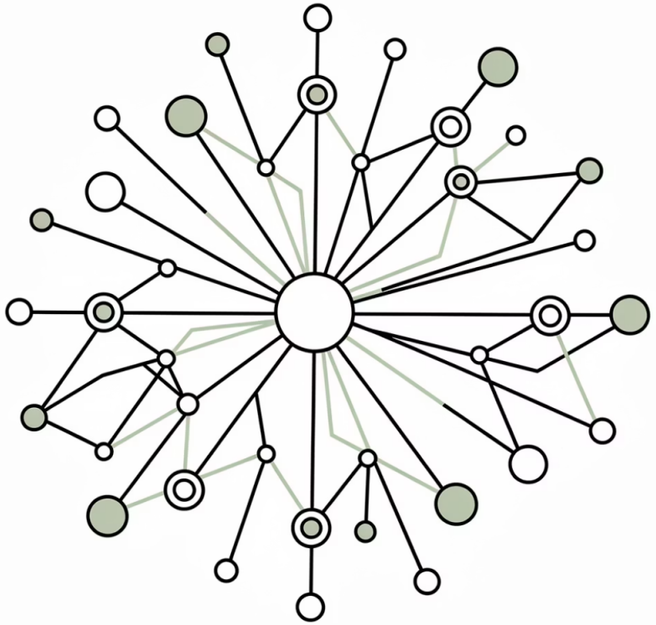
Арилжааны системүүд

Oracle, IBM DB2, MySQL нь холбоост өгөгдлийн санг үйлдвэрлэлийн чухал програмуудын стандарт болгосон

ACID зарчмууд

Гүйлгээний баталгаа нь найдвартай байдал, тогтвортой байдлыг хангаж, санхүү болон бизнесийн чухал системд зайлшгүй шаардлагатай болсон

Вэбийн хувьсгал: 1990–2000-аад он



Өгөгдлийн тэсрэлт өсөлт

Дэлхийн өргөн сүлжээний өсөлт нь урьд өмнө байгаагүй өгөгдлийн хэмжээг бий болгосон. Вэб сайтууд статик хуудаснаас дэлхий даяар сая сая зэрэг хэрэглэгчдэд үйлчилдэг динамик програм болон хувирсан.

Шинэ шаардлагууд гарч ирсэн

- Хурдан хөгжүүлэлтийн мөчлөгүүд схемийн уян хатан байдлыг шаардсан
- Дэлхийн хэрэглэгчдийн баазууд хэвтээ өргөтгөх чадварыг шаардсан
- Бодит цагийн харилцан үйлдэл миллисекундээс доош хариу цагийг шаардсан
- Төрөл бүрийн өгөгдлийн төрлүүд хатуу хүснэгтийн бүтцийг эсэргүүцсэн

Тогтвортой байдлыг хурдаас илүүд үзсэн уламжлалт харилцааны системүүд эдгээр шаардлагыг эдийн засгийн хувьд хангахад бэрхшээлтэй байсан.

NoSQL хувьсгал: 2000-2010-аад он



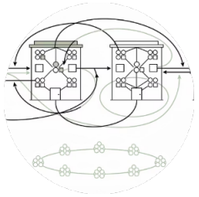
Баримт бичгийн сангууд

MongoDB болон CouchDB нь JSON-тэй төстэй уян хатан өгөгдлийн бүтцийг боломжтой болгож, хувьсан схем болон үүрлэсэн мэдээлэлд төгс тохирсон



Түлхүүр-утга сангууд

Redis болон DynamoDB нь микросекундын хугацаа шаардлагатай сесс, кэш хийх, бодит цагийн програмд зориулсан маш хурдан хайлт өгсөн



Баганын гэр бүл сангууд

Cassandra болон HBase нь цаг хугацааны цуваа өгөгдөл болон бичилт ихтэй ажлын ачаалалд шугаман хэвтээ өргөтгөх чадвартай тэргүүлсэн



График өгөгдлийн сангууд

Neo4j болон ArangoDB нь нийгмийн сүлжээ, зөвлөмж, залилан мэхлэлт илрүүлэх системд зориулж харилцааны асуулгыг оновчтой болгосон

ӨНӨӨГИЙН ШИЛДЭГ ПРАКТИК

Polyglot Persistence: 2020-аад он ба түүнээс хойш

Орчин үеийн архитектур нь нэг системд олон өгөгдлийн сангийн технологийг хүлээн зөвшөөрч, тодорхой хэрэглээний тохиолдол бүрт хамгийн оновчтой хэрэгслийг сонгож байна. Энэ хандлага нь гүйцэтгэл, хөгжүүлэгчийн бүтээмжийг дээд зэргээр нэмэгдүүлж, зардлын үр ашгийг сайжруулдаг.

Зөв хэрэгсэл, зөв ажил

PostgreSQL дээр санхүүгийн гүйлгээ, MongoDB дээр хэрэглэгчийн профайл, Redis дээр кэш хийх, Elasticsearch дээр хайлт—бүрэлдэхүүн хэсэг бүр өөрийн шаардлагад хамгийн тохирсон өгөгдлийн санг ашигладаг

Microservices архитектур

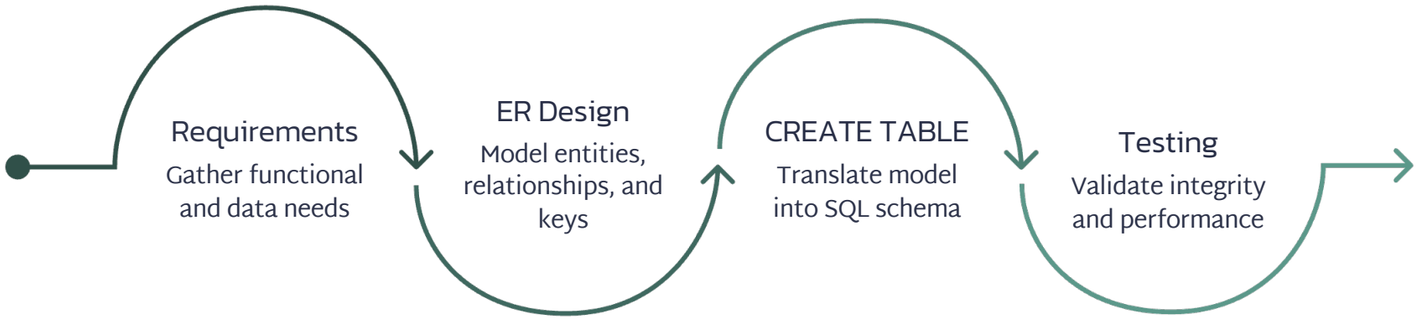
Үйлчилгээний хил хязгаар нь өгөгдлийн сангийн олон янз байдлыг боломжтой болгодог. Microservice бүр өөрийн өгөгдлийн санг эзэмшдэг бөгөөд тодорхой хандалтын загвар болон тогтвортой байдлын хэрэгцээнд үндэслэн технологи сонгодог

Үйл явдал удирдуулсан интеграци

Мессежийн дараалал болон үйл явдлын урсгал нь ялгаатай өгөгдлийн сангууд дээр өгөгдлийг синхрончлж, бие даасан байдал болон гүйцэтгэлийг хадгалахын зэрэгцээ тогтвортой байдлыг хангадаг

УЛАМЖЛАЛТ ХАНДЛАГА

Waterfall өгөгдлийн сангийн дизайны үйл явц



Уламжлалт waterfall хандлага нь өгөгдлийн сангийн дизайныг кодлох эхлэхээс өмнө дуусгах тодорхой үе шат гэж үздэг. Үе шат бүр дараагийнх эхлэхээс өмнө дуусах ёстой.

Үндсэн зарчмууд

- **Өгөгдөл бол тогтвортой хөрөнгө:** Схемүүд 5-10 жил үргэлжлэхээр зохион бүтээгдсэн
- **Эхлээд дизайн, дараа нь код:** Схемийн өөрчлөлт нь үнэтэй үйлдэл
- **Нормчлал нь алтан дүрэм:** Давхардлыг бүрэн арилгах
- **ACID давамгай байдал:** Өгөгдлийн бүрэн бүтэн байдал нь хамгийн чухал

Дизайн яагаад чухал вэ: *Түүхэн нөхцөл байдал*

1970–1990: Хадгалах нь үнэтэй байсан

1980 онд 1GB хадгалахад ойролцоогоор $200,000 USD зардал гардаг байсан. Өгөгдлийн давхардал нь шууд санхүүгийн алдагдлыг илэрхийлдэг байсан. Нормчлал нь эдийн засгийн хэрэгцээ байсан, зөвхөн дизайны давуу тал биш байсан.

Хадгалах зардал нь дизайныг удирддаг байсан: Давтагдсан байт бүр үйл ажиллагааны зардлыг мэдэгдэхүйц нэмэгдүүлдэг байсан. Босоо өргөтгөл нь цорын ганц сонголт байсан тул өгөгдлийн үр дүнтэй зохион байгуулалт нь системийн амьдрах чадварт чухал байсан.

2020-аад он: Хадгалах нь хямд

Өнөөдөр 1GB ойролцоогоор $0.02 USD болсон—сая дахин бууралт. Давхардал нь одоо тодорхой хувилбаруудад хүлээн зөвшөөрөгддөг бөгөөд гүйцэтгэлийн өсөлтөд денормализаци хийх боломжтой болгосон.

Гэхдээ сайн дизайн чухал хэвээр байна: Өгөгдлийн бүрэн бүтэн байдал, гүйлгээний аюулгүй байдал, нарийн асуулгын чадвар, стандартчилсан SQL нь олон програмд үнэ цэнэтэй хэвээр байна.

Үргэлжлэн оршиж байгаа уламжлалт давуу талууд

Өгөгдлийн бүрэн бүтэн байдлын хамгаалалт

Гадаад түлхүүрийн хязгаарлалт, шалгах хязгаарлалт, trigger нь бизнесийн дүрмийг өгөгдлийн сангийн түвшинд хэрэгжүүлж, буруу өгөгдлийн байдлаас урьдчилан сэргийлдэг

Гүйлгээний аюулгүй байдал

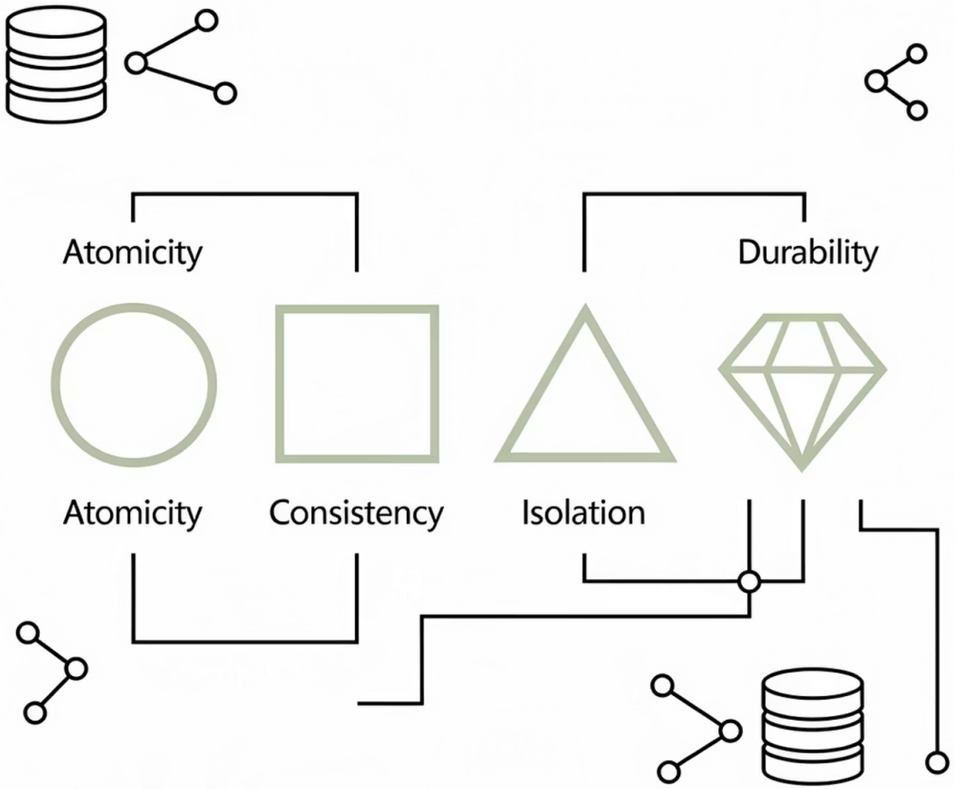
ACID баталгаа нь алдаа гарсан үед ч гэсэн найдвартай, тогтвортой үйлдлийг хангаж, санхүү болон чухал системд зайлшгүй шаардлагатай

Нарийн асуулгууд

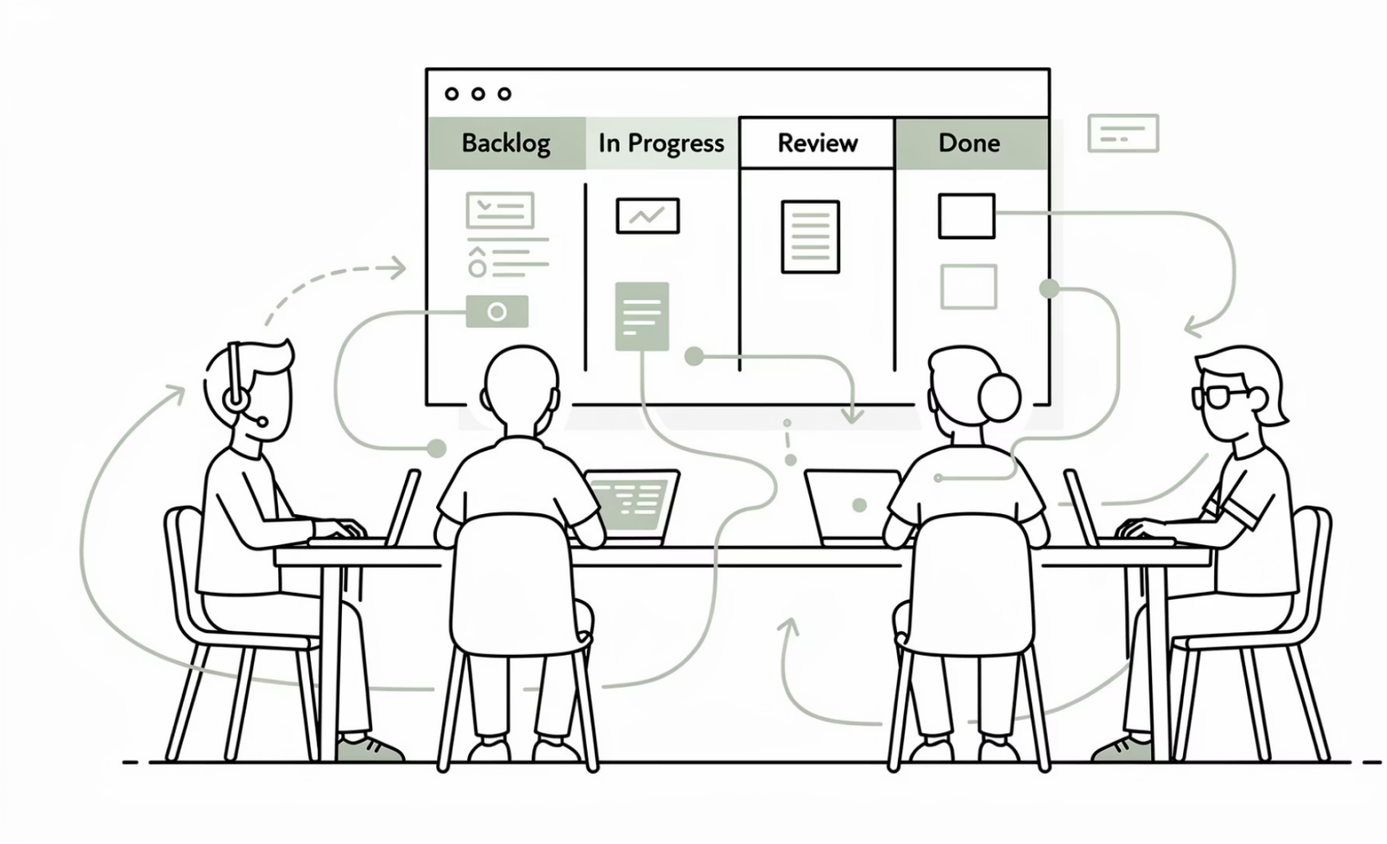
SQL-ийн хүчирхэг JOIN үйлдлүүд нь олон холбоотой схемд зарлагч синтаксээр нарийн аналитик асуулгыг боломжтой болгодог

Үйлдвэрлэлийн стандарт

SQL нь дэлхий даяар ойлгогддог, өргөнөөр баримтжуулагдсан бөгөөд хэдэн арван жилийн хэрэгсэл болон мэргэжлийн дэмжлэгтэй



Орчин үеийн шаардлага уламжлалыг эсэргүүцдэг



Шинэ бодит байдал

Өнөөгийн програмууд нь уламжлалт хандлагаар эдийн засгийн хувьд шийдвэрлэхэд бэрхшээлтэй шаардлагуудтай тулгардаг:

Хурдан хувьсал

Схемүүд долоо хоног бүр шинэ боломжуудыг хүлээн авах ёстой. Уламжлалт ALTER TABLE үйлдлүүд нь зогсолт болон нарийн төвөгтэй байдлыг үүсгэдэг.

Дэлхийн хэмжээ

Програмууд нь сая саяыг зэрэг үйлчилдэг. Босоо өргөтгөл нь физик хязгаарт хүрч байхад хэвтээ өргөтгөл нь харилцааны архитектурт сорилт болдог.

Өгөгдлийн олон янз байдал

Хэрэглэгчийн үүсгэсэн контент нь бүтцээрээ маш их ялгаатай. Хатуу схемүүд ийм уян хатан байдлыг гоёмсгоор загварчлаж чадахгүй.

Муу дизайн: Урьд болон одоо үр дагавар

Уламжлалт асуудал: Давхардал

```
-- Муу жишээ: Давтагдсан өгөгдөл
CREATE TABLE ORDERS (
  order_id INT PRIMARY KEY,
  order_date DATE,
  customer_name VARCHAR(100),    -- Давхардсан!
  customer_address VARCHAR(200), -- Давхардсан!
  customer_phone VARCHAR(20)     -- Давхардсан!
);
```

Асуудлууд: Үйлчлүүлэгчийн хаягийн өөрчлөлт нь бүх захиалгыг шинэчлэхийг шаарддаг. Нэг үйлчлүүлэгч захиалгууд дээр зөрүүтэй хаягтай байж болно. Хадгалах орон зайн үрэлгэн нь санхүүгийн хувьд чухал байсан.

Орчин үеийн асуудал: Хэт үүрлэлт

```
// MongoDB - Хэт үүрлэсэн бүтэц
{
  "order_id": "ORD001",
  "customer": {
    "name": "John Smith",
    "orders": [  // Хязгааргүй рекурс!
      {
        "order_id": "ORD001",
        "customer": {...}  // Рекурсив!
      }
    ]
  }
}
```

Асуудлууд: Баримт бичгүүд хэт том болж байна. Шинэчлэлтүүд нарийн төвөгтэй болж удааширдаг. 16MB баримт бичгийн хязгаар бодит хязгаарлалт болдог.

ӨГӨГДЛИЙН БҮРЭН БҮТЭН БАЙДАЛ

ACID vs BASE: Өрсөлдөгч философиуд

Уламжлалт ACID зарчмууд

Atomicity (Атом байдал): Бүх үйлдлүүд амжилттай болох эсвэл бүгд бүтэлгүйтдэг. Мөнгөний шилжүүлэг нь дебит болон кредит үйлдлүүд дуусахыг шаарддаг.

Consistency (Тогтвортой байдал): Өгөгдөл үргэлж хүчинтэй төлөвт үлдэнэ. Хязгаарлалт болон гадаад түлхүүрүүд өгөгдлийн сангаар автоматаар хэрэгждэг.

Isolation (Тусгаарлалт): Зэрэгцээ гүйлгээнүүд хооронд нь саад болохгүй. Олон хэрэглэгч аюулгүйгээр өгөгдлийг зэрэг өөрчилж чадна.

Durability (Бат бөх байдал): Баталгаажсан өгөгдөл хэзээ ч алга болохгүй. Цахилгааны тасалдал нь дууссан гүйлгээг эвдэж чадахгүй.

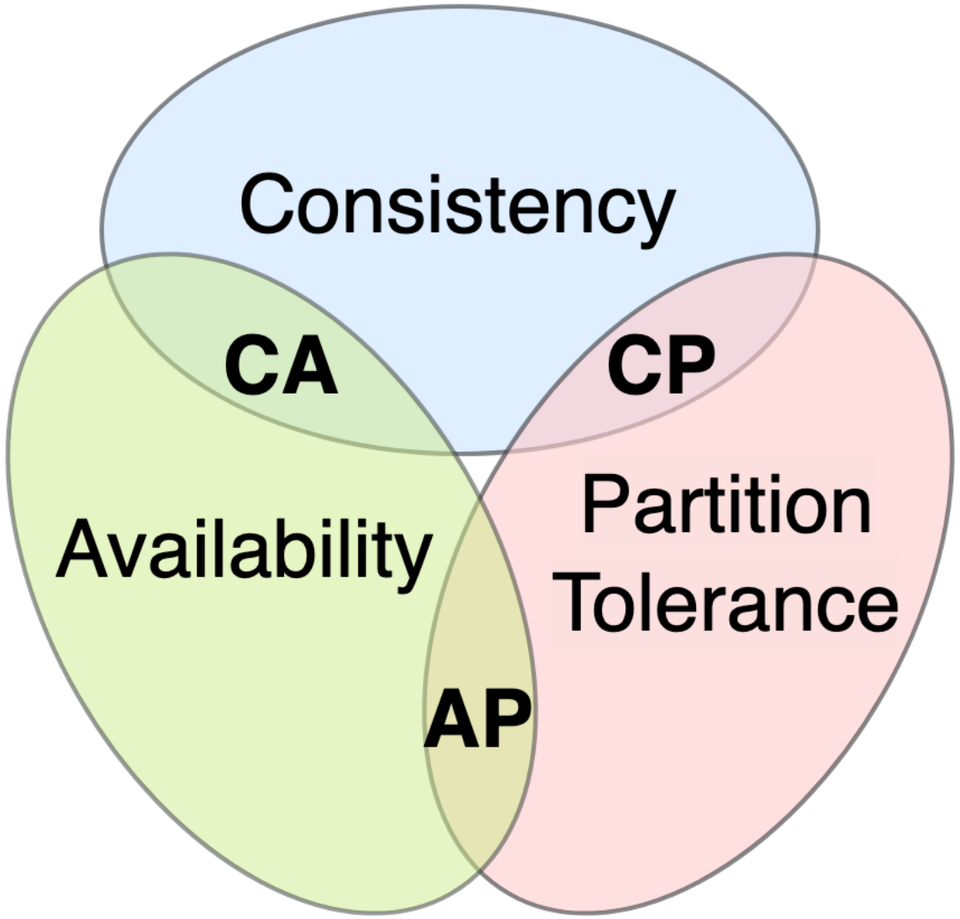
Орчин үеийн BASE зарчмууд

Basically Available (Үндсэндээ боломжтой): Систем үргэлж хариу өгдөг, хамгийн сүүлийн өгөгдөл биш ч гэсэн. Боломжит байдал нь төгс тогтвортой байдлаас давуу.

Soft state (Зөөлөн төлөв): Өгөгдөл түр зуур хувьсах боломжтой. Систем нь зангилаа хооронд шууд тогтвортой байдлыг баталгаажуулахгүй.

Eventually consistent (Эцэст нь тогтвортой): Цаг хугацаа өгвөл бүх зангилаа ижил төлөвт ирнэ. Тогтвортой байдал нь асинхрон байдлаар хүрдэг.

CAP теорем: Хоёрыг сонго



Үндсэн зөрчилдөөн

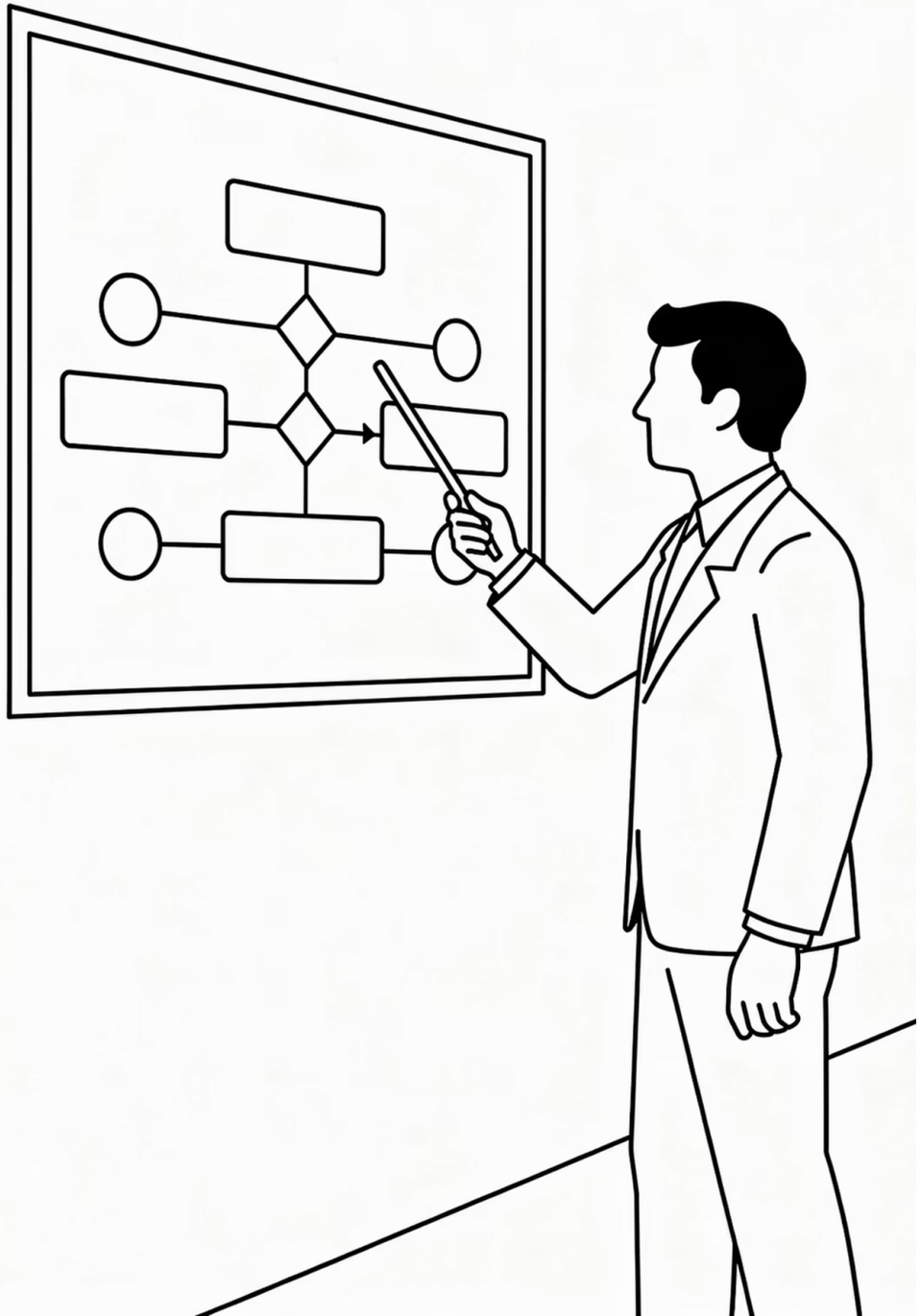
Eric Brewer-ийн CAP теорем (2000) нь тархсан системүүд гурван шинж чанарыг зэрэг баталгаажуулж чадахгүй гэдгийг нотолсон. Энэ нь гүн гүнзгий нөлөөтэй архитектурын шийдвэрийг шаарддаг.

Практик сонголтууд

CA (Consistency + Availability): PostgreSQL болон MySQL гэх мэт уламжлалт RDBMS. Банк болон санхүүгийн системд тохиромжтой, тогтвортой байдал нь үл тэвчих зүйл.

CP (Consistency + Partition tolerance): MongoDB болон HBase. Өгөгдлийн агуулах, нарийвчлал нь тасралтгүй боломжит байдлаас илүү чухал.

AP (Availability + Partition tolerance): Cassandra болон DynamoDB. Нийгмийн сүлжээ, ажиллах цаг болон хариу үйлдэл хамгийн чухал.



ТҮҮХЭН ЧУХАЛ ҮЙЛ ЯВДАЛ

Peter Chen-ий хувьсгал: ER загварчлал

1976 онд Peter Chen Entity-Relationship загварчлалыг танилцуулж, өгөгдлийн сангийн дизайнд хүмүүс ойлгож болох харааны хэлийг өгсөн. 50 жилийн турш ER диаграм нь өгөгдлийн сангийн бүтцийг харилцан ярианы алтан стандарт хэвээр байна.

> "Өгөгдөл нь төв байх ба програмууд түр зуурынх. Өгөгдлийн загварыг анхааралтай зохион бүтээвэл програмууд түүний эргэн тойронд амжилттай хувьсна. Өгөгдлийн дизайныг яаравал түүн дээр баригдсан програм бүр зовлон амсна."
>
> — Уламжлалт ER философи

ER бүрэлдэхүүн хэсгүүд: *Entity*

Уламжлалт тодорхойлолт

Entity нь асуудлын хүрээн дэх тогтвортой, бие даасан объектыг илэрхийлнэ. Ховорхон өөрчлөгддөг. Жишээнүүд нь Үйлчлүүлэгч, Бүтээгдэхүүн, Ажилтан—тодорхой хил хязгаар болон тогтвортой шинж чанартай ойлголтууд.

```
CUSTOMER хүснэгт:
- customer_id (PK)
- first_name
- surname
- registration_number
- address
- phone_number
```

Багана бүр урьдчилан тодорхойлогддог. Шинэ шинж чанар нэмэх нь бүх мөрт нөлөөлөх схемийн шилжүүлгийг шаарддаг.

Орчин үеийн NoSQL хандлага

Entity ойлголт уян хатал болсон. NoSQL дээрх баримт бичгүүд нь схемийн өөрчлөлтгүйгээр органик байдлаар хувьсаж болох уян хатан бүтэц.

```
// MongoDB - Уян хатан баримт бичиг
{
 "_id": "CUST001",
 "profile": {
 "firstName": "John",
 "contacts": [
 {"type": "mobile", "value": "123456"},
 {"type": "email", "value": "john@example.com"}
 ]
 },
 "metadata": {
 "createdAt": "2024-01-15",
 "customField": "ямар ч өгөгдөл" // Схемийн өөрчлөлт
хэрэггүй!
 }
}
```

Уламжлалт vs Орчин үе: Entity харьцуулалт



Схемийн тодорхойлолт шаардлагатай

Уламжлалт харилцааны өгөгдлийн сан нь бүрэн урьдчилан тодорхойлохыг шаарддаг. Өгөгдөл оруулахаас өмнө багана, төрөл, хязгаарлалт бүрийг зарласан байх ёстой.



Схемийн өөрчлөлт шилжүүлэлтгүйгээр

NoSQL дээр уян хатан байдал. Баримт бичгийн өгөгдлийн санд шинэ талбаруудыг шууд нэмэх боломжтой. Зөвхөн шинэ баримт бичигт талбар хэрэгтэй; одоо байгаа баримт бичгүүд шилжүүлэлтгүйгээр өөрчлөгдөөгүй хэвээр байна.



Уламжлалт схемийн өөрчлөлтүүд

Уламжлалт схемүүдийг өөрчлөх нь ALTER TABLE мэдэгдлүүдийг шаарддаг бөгөөд шилжүүлэлтийн үед хүснэгтүүдийг түгжиж, зогсолтыг үүсгэх боломжтой.

Шинж чанарууд: Хатуу байдлаас уян хатан байдал руу

Уламжлалт хандлага

Багана нь бүтцийг хатуу тодорхойлно. Шинж чанарын төрлүүд:

Энгийн: VARCHAR(50), INT, DATE—нэг атомын утга.

Нийлмэл: Задлаж болох (хаяг = хот + гудамж), олон баганууд эсвэл нормчлал шаарддаг.

Олон утгат: Тусдаа хүснэгт шаарддаг (утасны дугаар, имэйл хаяг) нормчлалыг хадгалахын тулд.

Гарсан: VIEW эсвэл тооцоолсон баганаар тооцоологддог.

```
CREATE TABLE STUDENT (
  student_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  registration CHAR(10) UNIQUE,
  birth_date DATE,
  CHECK (birth_date < CURRENT_DATE)
);
```

Орчин үеийн уян хатан байдал

Схемгүй баримт бичгүүд нь олон янзын өгөгдлийг байгалийн байдлаар хүлээн зөвшөөрдөг:

```
// MongoDB - Байгалийн үүрлэлт
{
  "_id": "STU001",
  "name": "John Smith",
  "birthDate": "2000-05-15",
  "hobbies": ["хөл бөмбөг", "шатар"],
  "address": {
    "city": "Лондон",
    "district": "Westminster"
  },
  "customAttributes": {
    "favouriteColour": "цэнхэр",
    "graduationYear": 2024
  }
}
```

Массив, үүрлэсэн объект, шинэ талбарууд схемийн өөрчлөлтгүйгээр нэмэгддэг.

Схемийн уян хатан байдал яагаад зайлшгүй болсон вэ

Agile хөгжүүлэлт

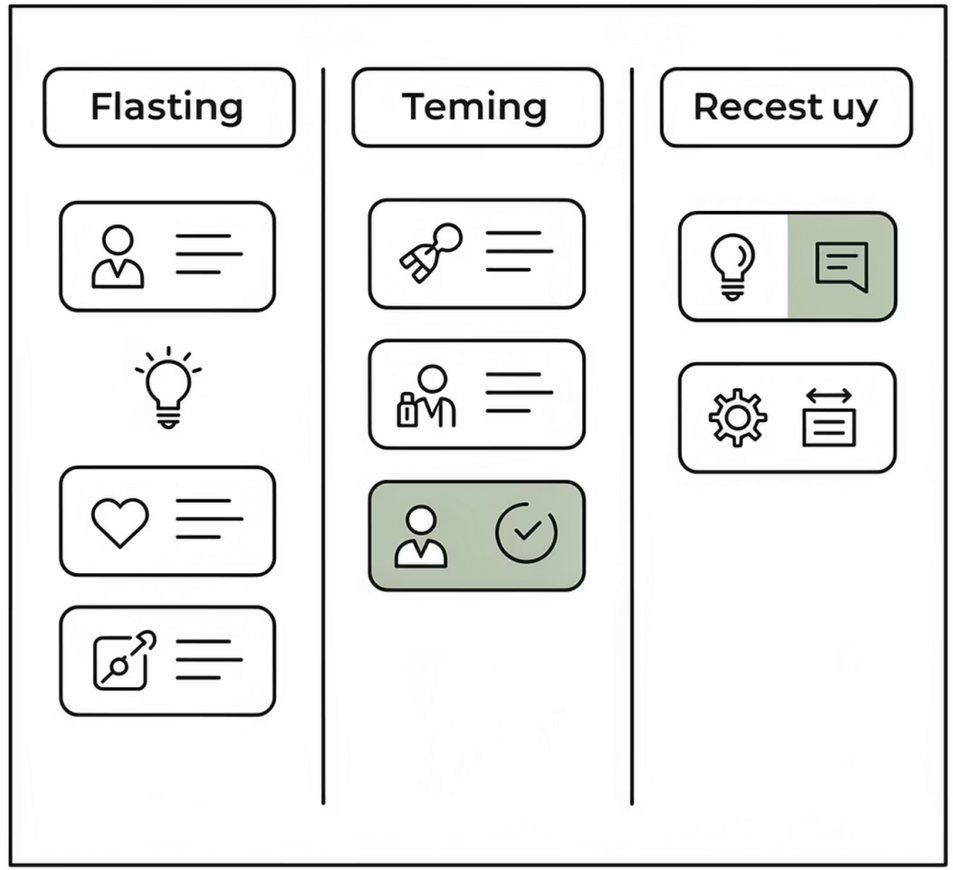
Спринтүүд долоо хоног бүр шинэ боломжуудыг өгдөг. Хатуу схемүүд саад болж, давталт болон инновацийг удаашруулдаг.

Startup соёл

"Хурдан хөдөлж, юмыг эвд." Бүтээгдэхүүн-зах зээлийн тохирол нь туршилт шаарддаг. Схемийн уян хатан байдал нь хурдан суралцахыг боломжтой болгодог.

Хэрэглэгчийн үүсгэсэн контент

Нийгмийн мэдээллийн хэрэгслийн пост маш их ялгаатай: текст, зураг, видео, байршил, санал асуулга, үйл явдал. Хатуу схемүүд ийм олон янз байдлыг гоёмсгоор хүлээн авч чадахгүй.



ХАРИЛЦАА ХОЛБОО

Харилцааг загварчлах: Статик vs Динамик

Entity хэрхэн холбогдох нь уламжлалт болон орчин үеийн хандлагын хооронд үндсэндээ ялгаатай. Сонгодог дизайн нь илэрхий, хэрэгжүүлсэн харилцаанд гадаад түлхүүрүүдийг ашигладаг. Орчин үеийн системүүд нь хандалтын загварт үндэслэн embedding болон referencing хооронд сонгодог.

1

Нэг–нэгт (1:1)

Уламжлалт дизайнд ховор ашиглагддаг. Жишээ: Иргэн ба Паспорт — иргэн бүр яг нэг паспорттай.

Нэг–олонт (1:N)

Хамгийн түгээмэл харилцаа. Жишээ: Тэнхим ба Оюутнууд — нэг тэнхим олон оюутан агуулна.

Олон–олонт (M:N)

Харилцааны дизайнд холбох хүснэгт шаарддаг. Жишээ: Оюутнууд ба Хичээлүүд — оюутнууд олон хичээлд элсэх; хичээлүүдэд олон оюутан байна.

Уламжлалт харилцаа: Гадаад түлхүүр

Нэг-олонт жишээ

```
CREATE TABLE DEPARTMENT (
  dept_id SERIAL PRIMARY KEY,
  dept_name VARCHAR(100)
);

CREATE TABLE STUDENT (
  student_id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  dept_id INT,
  FOREIGN KEY (dept_id)
    REFERENCES DEPARTMENT(dept_id)
);
```

Гадаад түлхүүрийн хязгаарлалт нь лавлагааны бүрэн бүтэн байдлыг автоматаар хэрэгжүүлдэг. Оюутнууд лавлаж байх үед тэнхимийг устгаж чадахгүй.

Олон-олонт жишээ

```
CREATE TABLE STUDENT (
 student_id INT PRIMARY KEY,
 name VARCHAR(100)
);

CREATE TABLE COURSE (
 course_id INT PRIMARY KEY,
 course_name VARCHAR(100)
);

CREATE TABLE ENROLLMENT (
 student_id INT,
 course_id INT,
 grade DECIMAL(3,2),
 PRIMARY KEY (student_id, course_id),
 FOREIGN KEY (student_id) REFERENCES STUDENT,
 FOREIGN KEY (course_id) REFERENCES COURSE
);
```

NoSQL харилцаа: Embedding vs Referencing

Embedding (Денормализаци)

```
// Хичээл нь бүх оюутныг агуулна
{
  "_id": "CS101",
  "courseName": "Програмчлалын үндэс",
  "students": [
    {
      "studentId": "STU001",
      "name": "John Smith",
      "grade": 85
    },
    {
      "studentId": "STU002",
      "name": "Jane Doe",
      "grade": 90
    }
  ]
}
```

Давуу тал: Нэг асуулга бүгдийг авна. Хурдан унших, холболтгүй.

Сул тал: Оюутны нэр шинэчлэх нь олон хичээлийн баримт бичгийг өөрчлөхийг шаарддаг. Өгөгдлийн давхардал.

Referencing (Нормчлал)

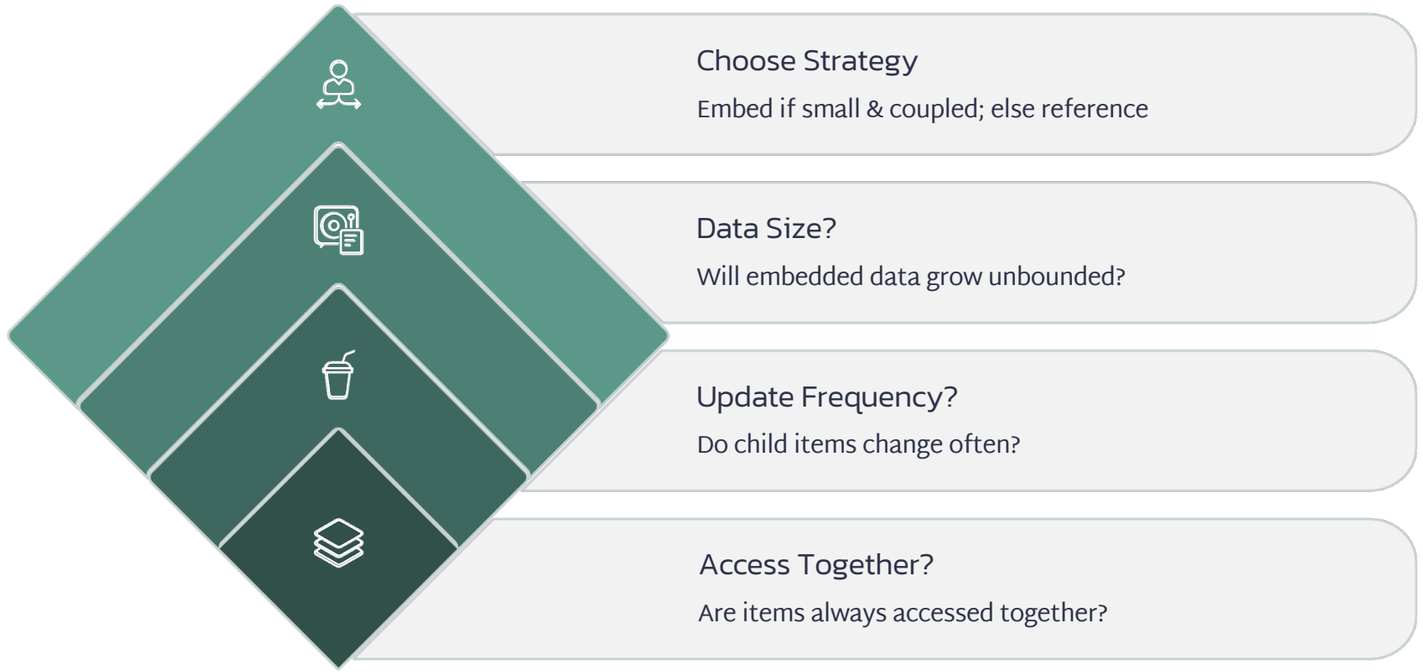
```
// Оюутны баримт бичиг
{
  "_id": "STU001",
  "name": "John Smith"
}

// Хичээлийн баримт бичиг
{
  "_id": "CS101",
  "courseName": "Програмчлалын үндэс",
  "studentIds": ["STU001", "STU002"]
}
```

Давуу тал: Өгөгдлийн тогтвортой байдал. Үнэний нэг эх үүсвэр. Шинэчлэлт энгийн.

Сул тал: Олон асуулга шаардлагатай. Програмын түвшний холболт хэрэгтэй.

Embedding болон Referencing хооронд сонгох



Стратегийн шийдвэрийн хүрээ

Embed хийх үед: Өгөгдөл үргэлж хамтдаа хандагддаг, харилцаа нь нэг-олонт, embedding хийсэн өгөгдөл хязгааргүй өсөхгүй. Жишээ: блог пост сэтгэгдэлтэй.

Reference хийх үед: Эд юмс бие даан шинэчлэгддэг, харилцаа нь олон-олонт, эсвэл embedding хийсэн өгөгдөл MongoDB-ийн 16MB баримт бичгийн хязгаараас давж болно. Жишээ: бүтээгдэхүүн ба ангилал.

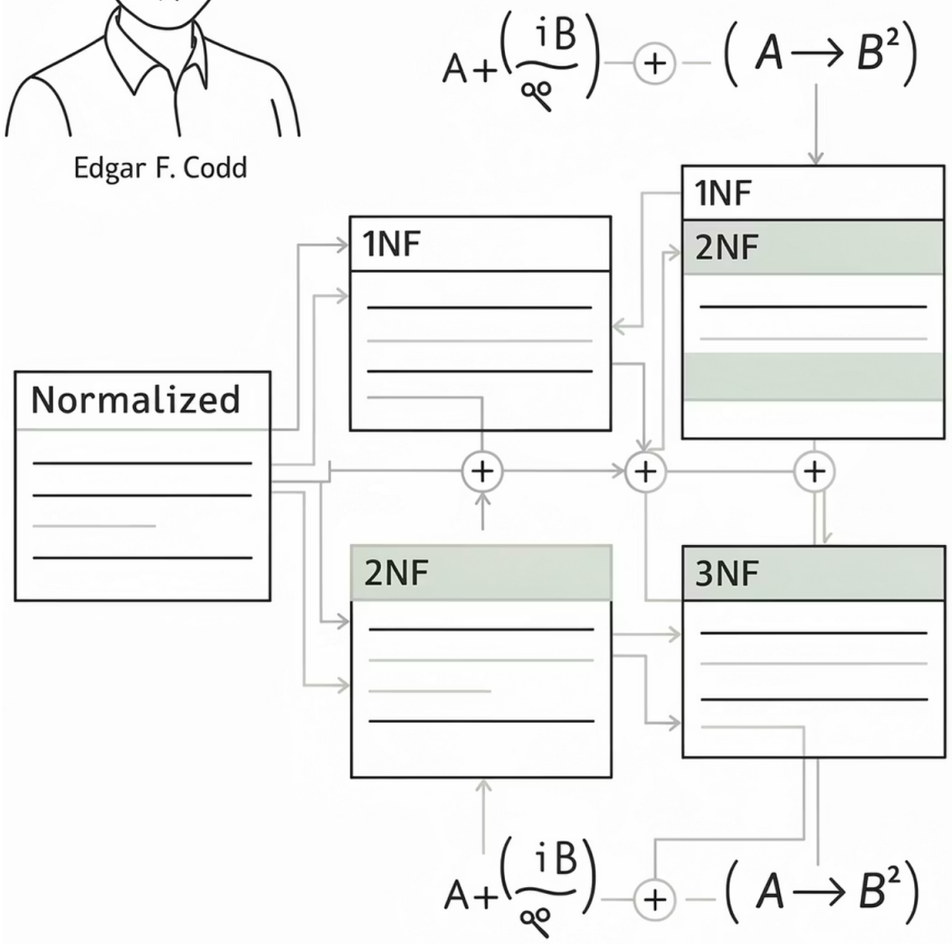
Энэ сонголт нь гүйцэтгэл, тогтвортой байдал, засвар үйлчлэгээнд гүнзгий нөлөөлнө. Универсал хариулт байхгүй—нөхцөл байдал нь хамгийн оновчтой хандлагыг тодорхойлдог.

НОРМЧЛАЛ

Нормчлал: Философи ба зорилго



Edgar F. Codd



> "Өгөгдлийн давхардал нь мэдээллийн систем дэх бүх эмх замбараагүй байдлын үндэс юм. Давтамжийг арилгавал тогтвортой байдал байгалийн дагуу дагадаг."
>
> — Edgar Codd, Нормчлалын эцэг (1970)

Codd-ын нормчлалын зарчмууд нь математик задлалаар давхардал болон шинэчлэлтийн хазайлтыг арилгах зорилготой байсан. 30 жилийн турш нормчлал нь эргэлзээгүй шилдэг практик байсан. Өнөөдөр бид гүйцэтгэлийн төлөө зориудаар денормализаци хийдэг—гэхдээ зөвхөн бид юуг золиослож байгаагаа ойлгох үед.

Нормчлалын зорилго: Урьд vs Одоо

Уламжлалт зорилго (1970–2000)

- БҮГД давхардлыг арилгаж үнэтэй хадгалах орон зайг хамгаалах
- Бүтцийн задлалаар шинэчлэлтийн хазайлтаас урьдчилан сэргийлэх
- Схемийн хязгаарлалтаар өгөгдлийн бүрэн бүтэн байдлыг хэрэгжүүлэх
- Холболтоор нарийн аналитик асуулгыг боломжтой болгох

Нормчлал нь эдийн засгийн болон техникийн хувьд зайлшгүй байсан.

Орчин үеийн харгалзан үзэх зүйлүүд (2000–Өнөөдөр)

- Денормализаци нь унших ихтэй ачаалалд ихэвчлэн зөв
- Унших vs бичих гүйцэтгэлийн зөрчилдөөнийг тэнцвэржүүлэх
- Өргөтгөх чадварын өсөлтөд давхардлыг хүлээн зөвшөөрөх
- Тархсан системд эцэст нь тогтвортой байдлыг хүлээн зөвшөөрөх

Нөхцөл байдал болон хандалтын загвар нь өнөөдөр нормчлалын шийдвэрийг удирддаг.

Эхний хэвийн хэлбэр (1NF): Атом байдал

Уламжлалт 1NF шаардлага

- Багана бүр атом (хуваагдашгүй) утга агуулна
- Багана бүр нэг өгөгдлийн төрөл хадгална
- Давтагдах баганын бүлэг байхгүй

Зөрчлийн жишээ

```
-- 1NF-д биш
STUDENT (
  student_id INT,
  name VARCHAR(50),
  phones VARCHAR(200)  -- "123456, 789012, 345678"
)
```

Олон утасны дугаарыг таслалаар тусгаарласан текст байдлаар хадгалах нь атом байдлыг зөрчинө.

Уламжлалт шийдэл

```
STUDENT (student_id, name)
PHONE (student_id FK, phone_number)
```

Тусдаа хүснэгт нь атом байдлыг хангаж, утасны дугаараар асуулга гаргахыг боломжтой болгоно.

NoSQL хандлага

```
{
  "_id": "STU001",
  "name": "John Smith",
  "phones": ["123456", "789012", "345678"]
}
```

Массив нь 1NF зөрчдөг боловч баримт бичгийн өгөгдлийн санд төгс хүлээн зөвшөөрөгдөх—бүр давуу тал. Зохих индексээр асуулга үр дүнтэй хэвээр байна.

Хоёрдугаар хэвийн хэлбэр (2NF): Хэсэгчилсэн хамааралууд

Уламжлалт 2NF дүрэм

1NF-д байх ёстой бөгөөд түлхүүр биш шинж чанар бүр нь бүхэл бүтэн анхдагч түлхүүрээс хамаарах ёстой, зөвхөн хэсгээс биш.

Зөрчлийн жишээ

```
ENROLLMENT (
  student_id,      -- PK хэсэг 1
  course_id,       -- PK хэсэг 2
  instructor_name, -- Зөвхөн course_id-с хамаарна!
  grade            -- Хоёулаас хамаарна (Зөв)
)
```

Багшийн нэр нь зөвхөн course_id-с хамаарч, нийлмэл түлхүүр дээр хэсэгчилсэн хамаарал үүсгэнэ.

Уламжлалт шийдэл

```
ENROLLMENT (student_id, course_id, grade)
COURSE (course_id, course_name, instructor_name)
```

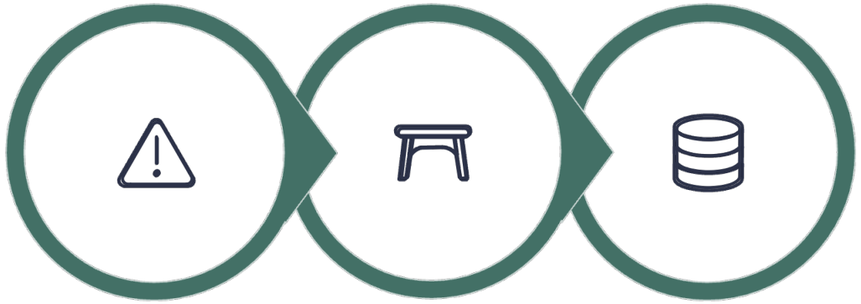
NoSQL "Зөрчил"

```
// Зориудын денормализаци
{
  "_id": ObjectId("..."),
  "studentId": "STU001",
  "courseId": "CS101",
  "courseName": "Програмчлал",
  "instructorName": "Доктор Смит",
  "grade": 85
}
```

Яагаад денормализаци хийх вэ? Элсэлтийн дэлгэрэнгүй мэдээлэл авахад олон хүснэгт холбох оронд нэг асуулга л хангалттай. Тайлан үүсгэх нь маш энгийн болно.

Зөрчилдөөн: Хэрэв Доктор Смит нэрээ өөрчилвөл бүх элсэлтийн баримт бичиг шинэчлэх шаардлагатай. Түүхэн өгөгдөлд (дүн) энэ нь хүлээн зөвшөөрөгдөх боломжтой—та цаг хугацааны агшинг хүлээн авч байна.

Гуравдугаар хэвийн хэлбэр (3NF): Дамжуулах хамааралууд



Уламжлалт 3NF тодорхойлолт

2NF-д байх ёстой бөгөөд дамжуулах хамааралууд байхгүй (A→B→C, A нь түлхүүр).

Зөрчлийн жишээ

```
STUDENT (
  student_id,    -- PK
  name,
  dept_id,       -- dept_id → dept_name
  dept_name      -- Дамжуулах хамаарал!
)
```

Тэнхимийн нэр нь тэнхимийн ID-с хамаарч, энэ нь оюутны ID-с хамаардаг—дамжуулах гинж.

Уламжлалт шийдэл

```
STUDENT (student_id, name, dept_id FK)
DEPARTMENT (dept_id, dept_name)
```

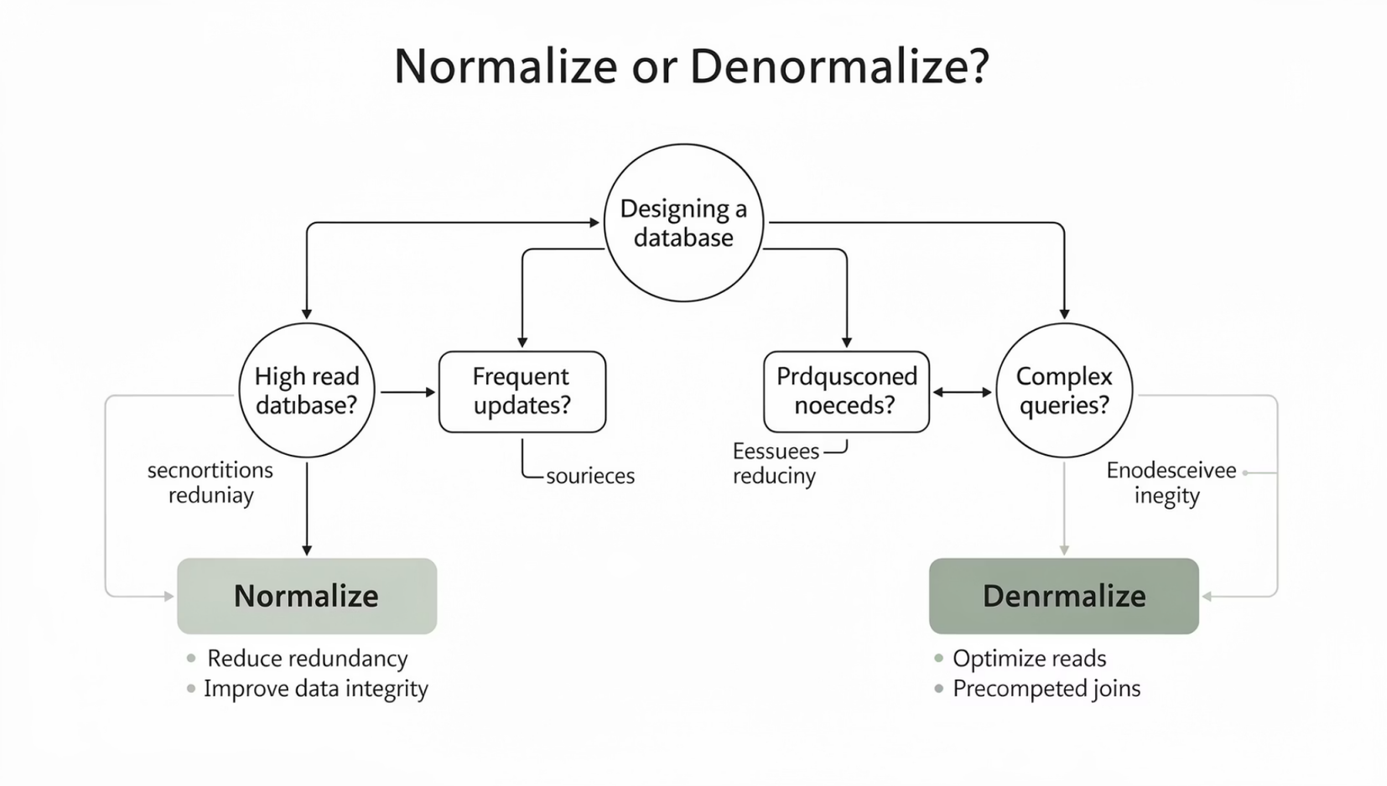
Задлалаар дамжуулах хамааралыг арилгана.

NoSQL хандлага

```
{
  "_id": "STU001",
  "name": "John Smith",
  "department": {
    "id": "DEPT-CS",
    "name": "Компьютерийн шинжлэх ухаан",
    "building": "5-р байр"
  }
}
```

Тэнхимийн дэлгэрэнгүй embedding хийх нь 3NF зөрчдөг боловч нэг асуулгаар профайл авахыг боломжтой болгоно. Унших ихтэй програмд энэ зөрчилдөөн нь хэрэглэгчийн туршлагыг сайжруулна.

Хэзээ нормчлах vs денормализаци хийх



Шийдвэрийн хүрээ

Нормчлах (3NF) үед: Тогтвортой байдал хамгийн чухал, гүйлгээ чухал, бичих үйлдлүүд давамгайлна. Жишээ: банк, ERP, CRM системүүд.

Денормализаци хийх үед: Унших гүйцэтгэл нь бичих энгийн байдлаас илүү чухал, холболт үнэтэй болж, эцэст нь тогтвортой байдал хүлээн зөвшөөрөгдөнө. Жишээ: нийгмийн мэдээлэл, контент хүргэх, аналитик.

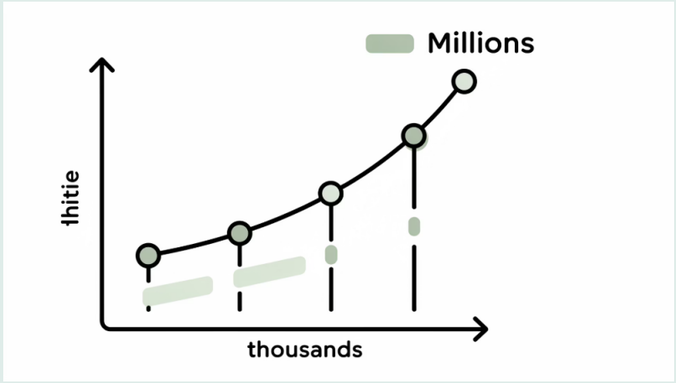
Эрлийз хандлага: Олон системүүд түүхэн өгөгдөлд агшин зурган дээр суурилсан денормализаци ашигладаг бол одоогийн төлөвийг нормчлагдсан байлгадаг. Цахим худалдааны захиалгын түүх нь сонгодог жишээ юм.

NOSQL ХУВЬСГАЛ

NoSQL яагаад гарч ирсэн вэ: Web 2.0 сорилт

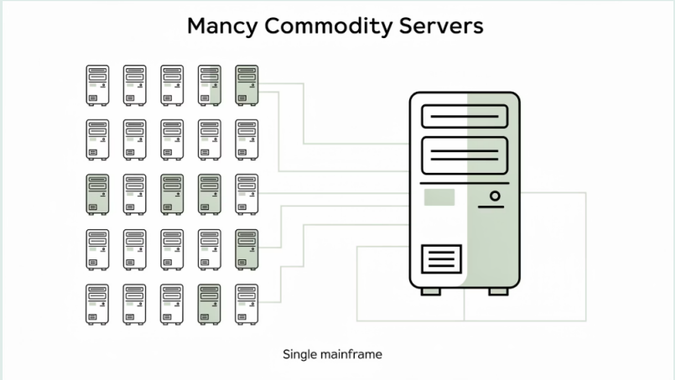
Хэрэглэгчийн хэмжээний тэсрэлт

Олон мянгаас хэдэн зуун сая зэрэг хэрэглэгчид жилүүдийн дотор. Уламжлалт босоо өргөтгөл эдийн засгийн хувьд гишгэх чадваргүй байсан.



Зардлын давуу тал

Хямд серверүүдээр хэвтээ өргөтгөл нь том босоо машин худалдаж авахаас хамаагүй хямд болсон. Мянга мянган хямд сервер нэг үнэтэй mainframe-ийг ялдаг.



Схемийн уян хатан байдлын хэрэгцээ

Нийгмийн платформууд боломжуудыг байнга хувьсгасан. ALTER TABLE бүр зогсолт үүсгэсэн. Схемийн уян хатан байдал нь үйл ажиллагааны хэрэгцээ болсон.



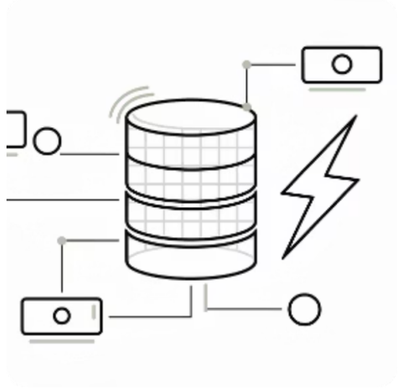
NoSQL өгөгдлийн сангийн төрлүүд ба хэрэглээ



Баримт бичгийн сангууд

Жишээ: MongoDB, CouchDB

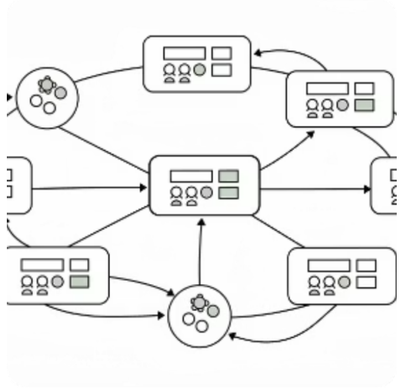
Хамгийн сайн нь: Контентын менежмент, хэрэглэгчийн профайл, бүтээгдэхүүний каталог, өгөгдөл нь үүрлэсэн шаталсан бүтэцтэй байгалийн баримт бичгийн бүтэцтэй газар



Түлхүүр–утга сангууд

Жишээ: Redis, DynamoDB

Хамгийн сайн нь: Кэш хийх, сессийн хадгалалт, бодит цагийн лидерборд, микросекундын хугацаа болон энгийн хайлт шаардлагатай газар



Баганын гэр бүл сангууд

Жишээ: Cassandra, HBase

Хамгийн сайн нь: Цаг хугацааны цуваа өгөгдөл, үйл явдлын бүртгэл, IoT мэдрэгчийн өгөгдөл, бичилт ихтэй ажлын ачаалал болон шугаман өргөтгөх чадварын хэрэгцээтэй газар

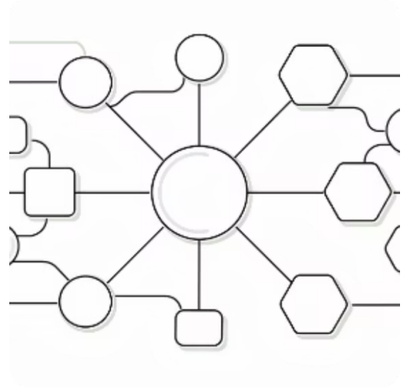


График өгөгдлийн сангууд

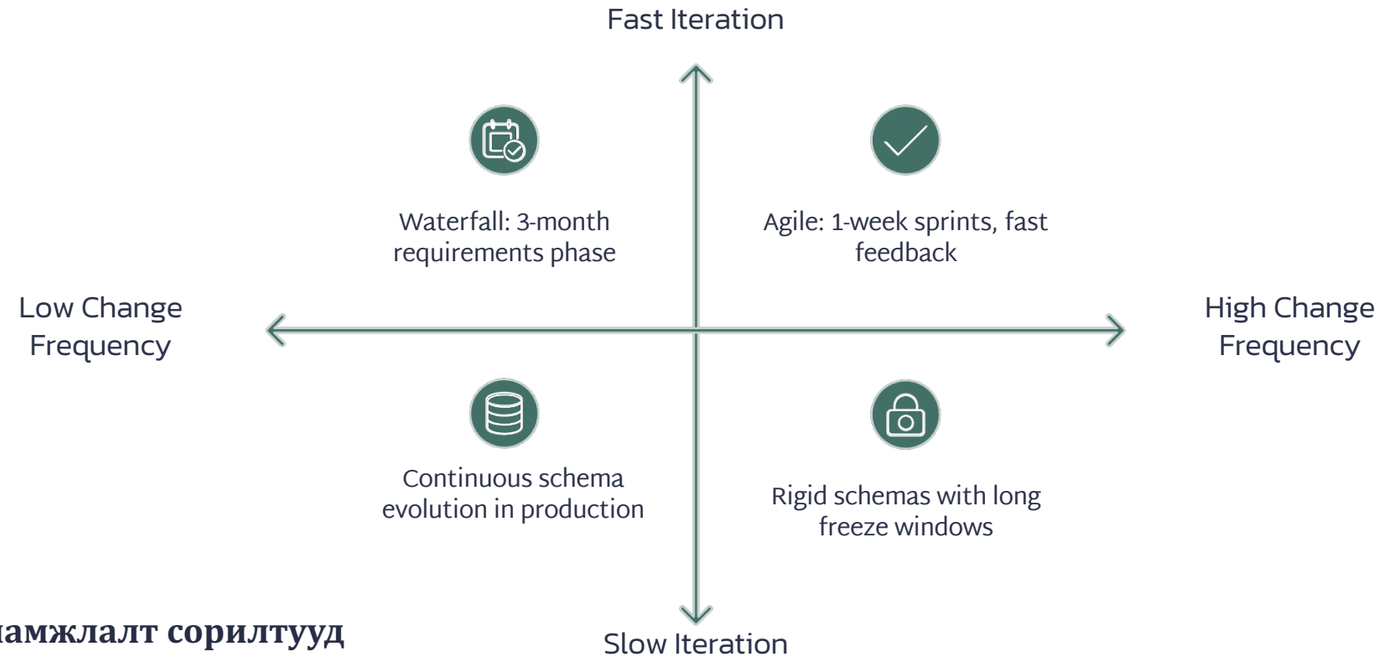
Жишээ: Neo4j, ArangoDB

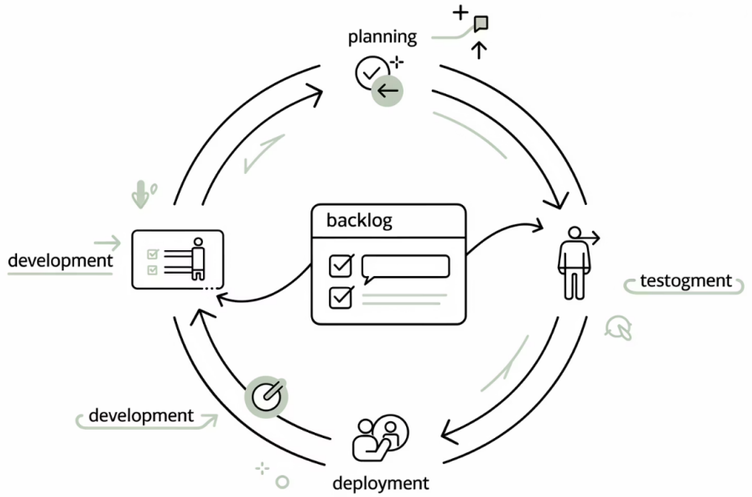
Хамгийн сайн нь: Нийгмийн сүлжээ, зөвлөмжийн хөдөлгүүр, залилан мэхлэлт илрүүлэх, харилцаа нь нэгдүгээр зэрэглэлийн иргэн байдаг газар

Agile хөгжүүлэлт ба схемийн хувьсал

Agile уян хатан байдлыг шаарддаг

Орчин үеийн хөгжүүлэлтийн аргачлалууд хурдан давтдаг. Боломжууд нь хэрэглэгчийн санал хүсэлтэд үндэслэн хэдэн өдрийн дотор хувьсдаг, сар биш. Өгөгдлийн сангийн схемүүд энэ хурдыг тохируулах ёстой.





Орчин үеийн бодит байдал

Спринт бүр шинэ өгөгдлийн шаардлага нэвтрүүлж болно. NoSQL-ийн схемийн уян хатан байдал нь органик хувьслыг боломжтой болгодог—одоо байгаа өгөгдлийг шилжүүлэлгүйгээр шинэ баримт бичигт шинэ талбарууд гарч ирнэ. Энэ нь тасралтгүй хүргэлтийн практиктай төгс нийцдэг.

Уламжлалт сорилтууд

Waterfall дизайн нь урьдчилан бүрэн тодорхойлолтыг шаарддаг. ER диаграмууд кодлох эхлэхээс өмнө эцэслэгдсэн байсан. Схемийн өөрчлөлтүүд ховор, төлөвлөгдсөн үйл явдал байсан бөгөөд өргөн турших болон шилжүүлэх скрипт шаарддаг байсан.

1

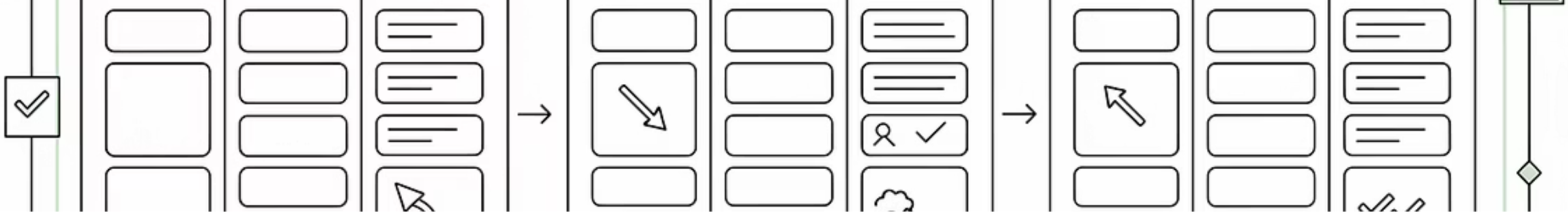
Waterfall

3 сарын шаардлагын үе шат, схемийн хөлдөөсөн цонх

2

Agile

1 долоо хоногийн спринт, үйлдвэрлэлд тогтмол схемийн хувьсал



Схемийн хувьслын жишээ: Agile спринтүүд

1

Спринт 1: MVP эхлүүлэх

```
{
  "_id": "USER001",
  "username": "john_smith",
  "email": "john@example.com"
}
```

Зайлшгүй хэрэглэгчийн өгөгдөлтэй хамгийн бага амьдрах чадвартай бүтээгдэхүүн.

2

Спринт 2: Нийгмийн боломжууд

```
{
  "_id": "USER001",
  "username": "john_smith",
  "email": "john@example.com",
  "followers": ["USER002", "USER003"],
  "bio": "Програм хангамжийн инженер"
}
```

Схемийн шилжүүлэлтгүйгээр шинэ талбарууд нэмэгдсэн. Хуучин баримт бичгүүд хүчинтэй хэвээр байна.

3

Спринт 3: Тоглоомжуулалт

```
{
  "_id": "USER001",
  "username": "john_smith",
  "email": "john@example.com",
  "followers": ["USER002", "USER003"],
  "bio": "Програм хангамжийн инженер",
  "gamification": {
    "points": 1250,
    "level": 5,
    "badges": ["эрт хэрэглэгч"]
  }
}
```

Үүрлэсэн объектууд нарийн төвөгтэй шинэ боломжуудыг жигд хүлээн авна.

Уламжлалт RDBMS-д спринт бүр ALTER TABLE мэдэгдлүүдийг шаарддаг бөгөөд зогсолт үүсгэж, буцах төлөвлөгөө шаарддаг байсан. NoSQL энэ үрэлтийг арилгана.

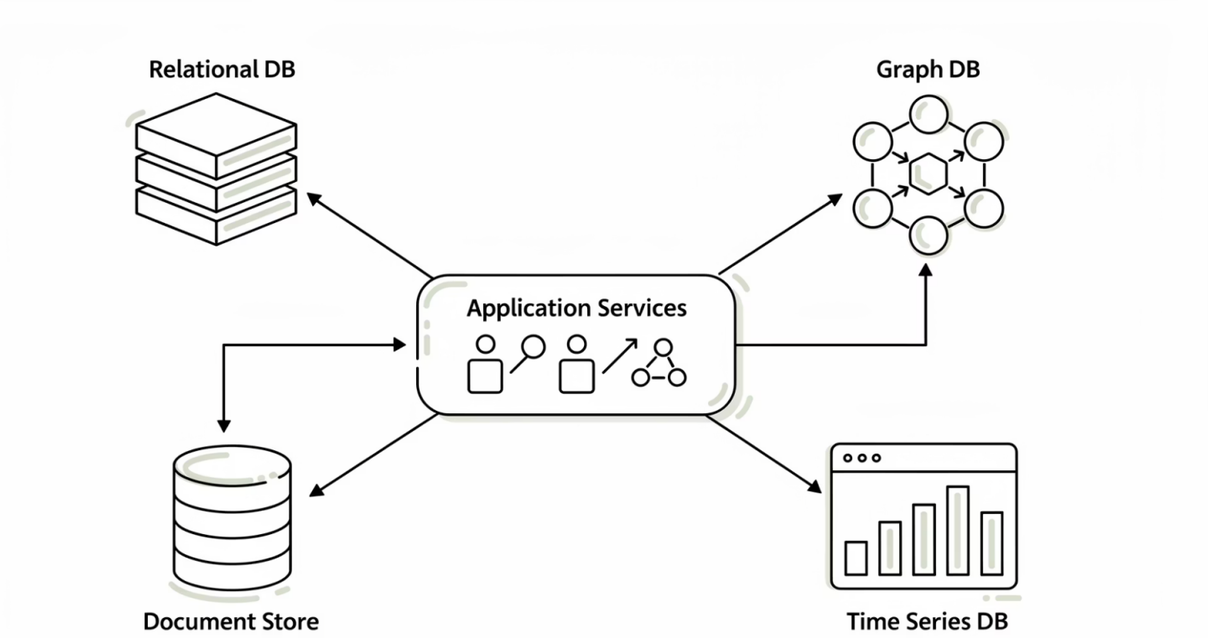
ОРЧИН ҮЕИЙН АРХИТЕКТУР

Polyglot Persistence: Ажил бүрт зөв хэрэгсэл

Стратегийн өгөгдлийн сангийн сонголт

Орчин үеийн системүүд "SQL эсвэл NoSQL?" гэж асуухгүй. Тэд "энэ тодорхой бүрэлдэхүүн хэсгийн шаардлагад хамгийн сайн үйлчилдэг аль өгөгдлийн сангийн төрөл вэ?" гэж асуудаг.

- **Хэрэглэгчийн бүртгэл:** PostgreSQL нь баталгаажуулалт болон эрхийн олголтод ACID баталгааг өгнө
- **Бүтээгдэхүүний каталог:** MongoDB нь схемийн толгой өвдөлтгүйгээр янз бүрийн бүтээгдэхүүний шинж чанаруудыг хүлээн авна
- **Худалдан авах сагс:** Redis нь түр зуурын сессийн өгөгдөлд санах ойн хурдыг санал болгоно
- **Хайлт:** Elasticsearch нь хамаарлын зэрэглэлтэй бүтэн текстийн асуулгад тэргүүлдэг
- **Зөвлөмжүүд:** Neo4j-ийн график асуулгууд холбоотой бүтээгдэхүүнийг үр дүнтэй олно



Дүгнэлт: Нарийн төвөгтэй байдлыг уян хатан хүлээн авах

Уламжлалт хүч чадал үргэлжилнэ

ACID гүйлгээ, өгөгдлийн бүрэн бүтэн байдал, SQL-ийн илэрхийлэх хүч нь санхүү, үйлдвэрлэл, чухал системд үнэ цэнэтэй хэвээр байна

Орчин үеийн уян хатан байдал хэмжээг боломжтой болгоно

NoSQL өгөгдлийн сангууд нь хэвтээ өргөтгөх чадвар, схемийн хувьсал, орчин үеийн вэб программд зайлшгүй гүйцэтгэлийн шинж чанаруудыг нээнэ

Polyglot Persistence стандарт болно

Амжилттай системүүд нь олон өгөгдлийн сангийн төрлүүдийг хослуулж, шашны давуу байдал гэхээсээ илүү тодорхой шаардлагад үндэслэн тус бүрийг сонгодог

Нөхцөл байдал шийдвэрийг удирддаг

Аль ч хандлага нь универсал байдлаар давуу биш. Зөрчилдөөн, хандалтын загвар, тогтвортой байдлын шаардлага, өргөтгөх чадварын хэрэгцээг ойлгох нь оновчтой сонголтуудыг удирддаг

Өгөгдлийн сангийн дизайн нь зөвхөн техникийн хэрэгжилт биш—энэ бол бизнесийн шаардлага, хэрэглэгчийн туршлага, системийн хувьслыг тэнцвэржүүлдэг стратегийн сэтгэлгээ юм. Тэргүүлэх системүүдийг архитектурчлахын тулд уламжлалт болон орчин үеийн хандлагуудыг эзэмш.